



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS
CURSO DE GRADUAÇÃO EM SISTEMAS DE INFORMAÇÃO

TRABALHO PRÁTICO I
ALGORITMOS UNION-FIND E CONECTIVIDADE DINÂMICA

CRATEÚS, CE
IARA RODRIGUES DE FARIAS
ANTONIA FLÁVIA MAGALHÃES MENDES

TRABALHO PRÁTICO I
ALGORITMOS UNION-FIND E CONECTIVIDADE DINÂMICA

**Relatório apresentado como prática para
aprendizagem na disciplina de Algoritmos
de Complexidade Computacional**

Professor: Rafael Martins Barros

CRATEÚS, CE
2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. MÉTODO EXPERIMENTAL.....	4
2.1 Linguagem e ambiente de execução.....	4
2.2 Implementações desenvolvidas.....	5
2.3 Instâncias de teste.....	5
2.4 Métricas coletadas.....	6
2.5 Procedimento de medição.....	6
Comando de execução utilizado:.....	7
2.6 Decisões de implementação e dificuldades práticas.....	7
3. RESULTADOS.....	7
3.1 Tabelas de tempo de execução.....	7
Tabela 1 - Resumo de tempo por algoritmo e instância.....	7
3.1.1 Medições adicionais.....	8
3.2 Gráficos de custo por conexão.....	8
3.3 Gráficos de tempo por tamanho da instância.....	12
3.4 Critério de consolidação dos resultados.....	14
4. DISCUSSÃO.....	14
4.1 Desempenho do Quick-Find em entradas grandes.....	14
4.2 Como o Quick-Union modifica a estrutura de dados.....	14
4.3 Ganho do Quick-Union Ponderado.....	14
4.4 Confirmação das previsões teóricas.....	15
4.5 Limitações do estudo.....	15
5. CONCLUSÃO.....	16
6. REFERÊNCIAS.....	16

1. INTRODUÇÃO

O problema de conectividade dinâmica consiste em manter, de forma eficiente, componentes conectados de um conjunto de N elementos, inicialmente disjuntos, com suporte às operações $\text{find}(p)$, $\text{union}(p, q)$ e $\text{connected}(p, q)$.

Neste trabalho, foram implementadas e analisadas experimentalmente três versões da estrutura Union-Find:

1. **Quick-Find**
2. **Quick-Union**
3. **Quick-Union Ponderado**

O objetivo deste trabalho foi comparar, na prática, o desempenho desses algoritmos em instâncias de diferentes escalas e verificar se os resultados obtidos acompanham o que a teoria indica.

Em síntese teórica vista em aula:

- **Quick-Find**: Operação find em tempo constante e union com custo linear em N , devido a possível varredura completa do vetor id .
- **Quick-Union**: Representação por árvores, com uniões por ligação entre raízes.
- **Quick-Union Ponderado**: Estratégia de balanceamento por tamanho, reduzindo a altura das árvores e, consequentemente, o custo médio das operações.

2. MÉTODO EXPERIMENTAL

2.1 Linguagem e ambiente de execução

Linguagem utilizada: - Python 3.12.10

Sistema operacional: - Windows 11 PRO

Hardware:

Processador: Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz (2.30 GHz)

Memória RAM: 16GB

Armazenamento: SSD 238 GB

Ambiente de execução:

Terminal: Git Bash

Dependências principais: matplotlib, pandas

*Para manter a reprodutibilidade, os experimentos foram executados com o mesmo interpretador Python e com parâmetros fixos dentro de cada lote.

2.2 Implementações desenvolvidas

As implementações foram organizadas no projeto conforme abaixo:

src/quick_find.py

src/quick_union.py

src/ponderado_quick_union.py

Todas as implementações foram instrumentadas para coletar:

custo_i: Número de acessos ao vetor id para processar a i-ésima conexão

total_acessos: Número acumulado de acessos ao vetor id até a i-ésima conexão

custo_medio_i = $\text{total_i} / i$

Observação metodológica:

O total de acessos usado na análise exclui o custo de inicialização da estrutura, considerando somente o processamento das conexões de entrada.

2.3 Instâncias de teste

Foram utilizadas instâncias de referência e instâncias adicionais:

- **TinyUF**

- **MediumUF**

- **LargeUF**

- **teste_10000**

- **teste_50000**

- **teste_100000**

- teste_500000

As instâncias adicionais seguem padrão de conexões:

N

0 1

0 2

0 3

...

2.4 Métricas coletadas

As métricas utilizadas foram:

1. Tempo total de execução por experimento
2. Média e desvio padrão do tempo (com repetições)
3. custo_i (por conexão)
4. total_i e custo medio_i = total_i / i

2.5 Procedimento de medição

Para cada par (algoritmo, instancia), foi adotado um número de repetições definido por lote de execução, registrando-se:

- Tempo de cada repetição
- Tempo médio
- Desvio padrão

Além disso, para cada conexão processada, foram registrados:

- custo_i
- total_i
- custo_medio_i

Comando de execução utilizado:

```
`python scripts/rodar_todos_experimentos.py --data-dir data_lote_3 --results-dir results --plots-dir plots --runs 1 --quick-find-runs 1 --quick-find-max-n 0 --quick-union-max-n 100000`
```

No lote atual (data_lote_3), os parametros foram definidos para priorizar viabilidade computacional em instâncias grandes: Quick-Find foi desabilitado para esse lote, e Quick-Union foi limitado a $N \leq 100000$.

2.6 Decisões de implementação e dificuldades práticas

Durante o desenvolvimento, algumas decisões práticas foram necessárias para que os testes terminassem em tempo razoável com o processador da máquina:

- Separação das execuções em lotes (data_lote_1, data_lote_2, data_lote_3), em vez de tentar processar todas as instâncias pesadas em uma única rodada;
- Definição de limites para pular combinações muito custosas em determinados lotes (por exemplo, Quick-Find em N alto).

Essa medida foi tomada, pois para entradas muito grandes o projeto ficava em loop quase infinito e pode levar muito tempo para o processamento e ocasionando travamento na máquina.

Essa estratégia deixou o processo mais controlável, mas exigiu cuidado na comparação entre lotes, já que nem todos foram executados com a mesma configuração.

3. RESULTADOS

3.1 Tabelas de tempo de execução

Tabela 1 - Resumo de tempo por algoritmo e instância

Algoritmo	Instância	N	Runs	Tempo médio (seg)	Desvio padrão (seg)
Quick-Find	teste_100000	100000	1	1271.827457	0.000000

Quick-Union	teste_100000	100000	1	0.215212	0.000000
Quick-Union Ponderado	teste_500000	500000	1	1.288780	0.000000

Fonte: Resultados experimentais das autoras

Observação: No estado atual, o arquivo results/resumo_tempos.csv contém as 3 medições do último lote executado (data_lote_3). Como runs=1, o desvio padrão aparece como 0.0 nessas linhas.

3.1.1 Medições adicionais

Tabela 2 - Medições coletadas em execuções anteriores

Algoritmo	Instância	Operações	Runs	Tempo médio (s)	Desvio Padrão (s)	Custo médio final
Quick-Find	teste_10000	9999	10	57.78857 1	29.346822	15002.0000
Quick-Union	teste_10000	9999	01	11.51725 1	0.000000	5002.0000
Quick-Union-Ponderado	teste_10000	9999	10	0.023492	0.002519	3.0000

Nota: os valores da Tabela 2 tem caráter complementar e foram mantidos como histórico, sem compor o resumo do lote corrente (data_lote_3).

3.2 Gráficos de custo por conexão

Para cada par algoritmo/instância, foi gerado um gráfico com:

- Eixo X: número de conexões processadas
- Eixo Y: acessos ao vetor id
- Linha cinza: custo_i
- Linha vermelha: custo medio_i

Figura 01 - [Quick-find Tiny UF custos]

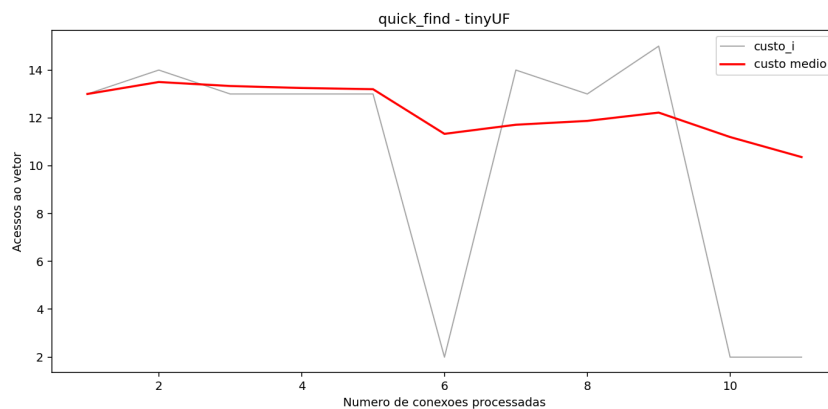


Figura 2 - [Quick find mediumUF_custos]

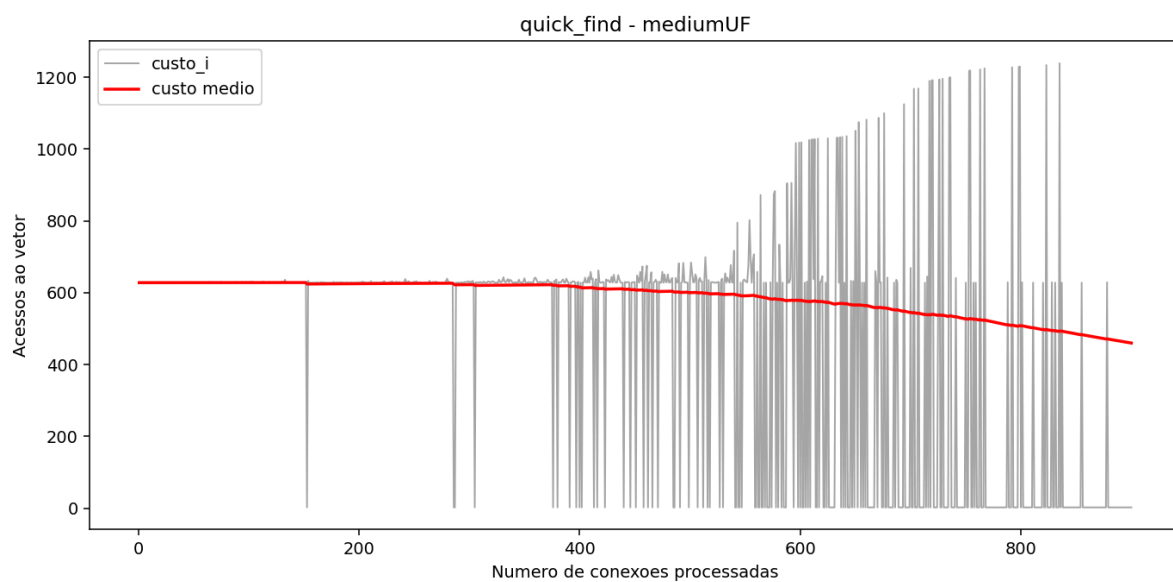


Figura 3 - [Quick_find_teste_10000_custos]

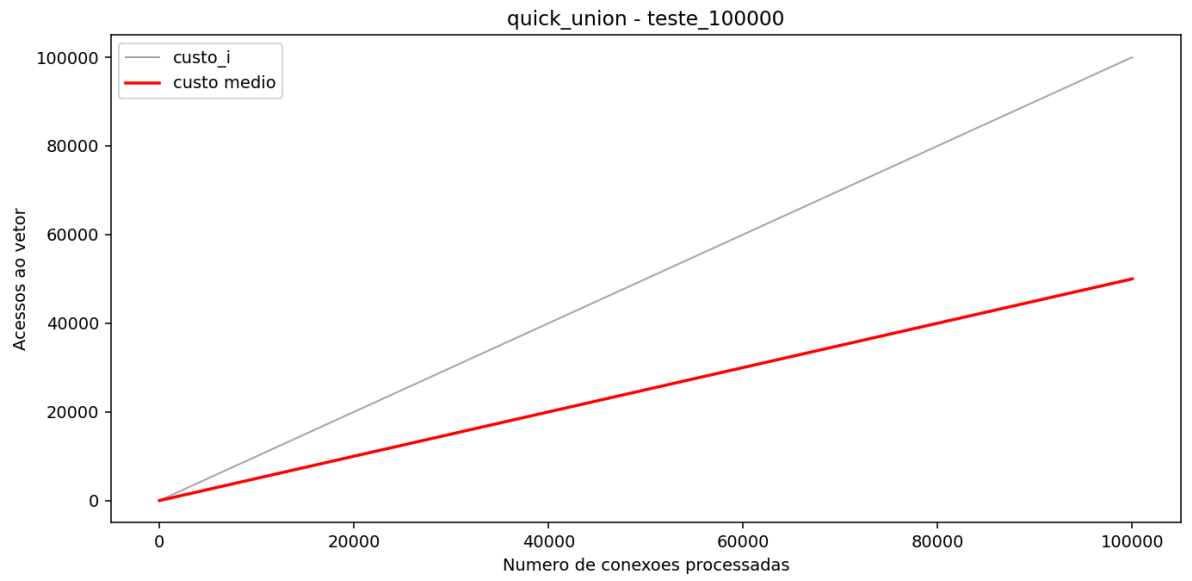


Figura 4 - [Quick_union_tinyUF_custos]

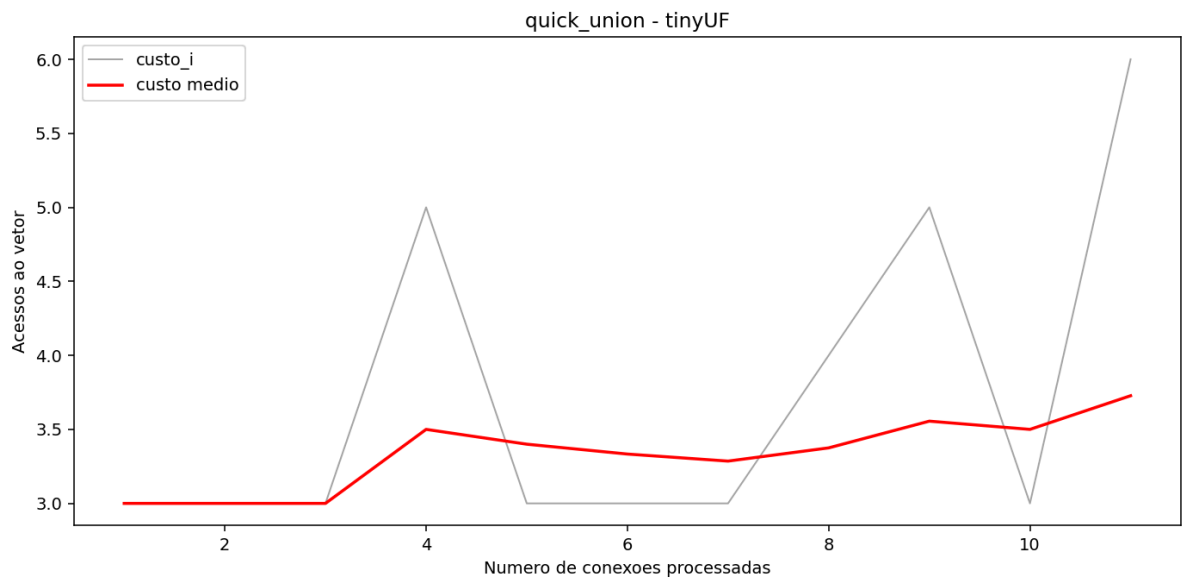


Figura 5 - [quick_union_mediumUF_custos]

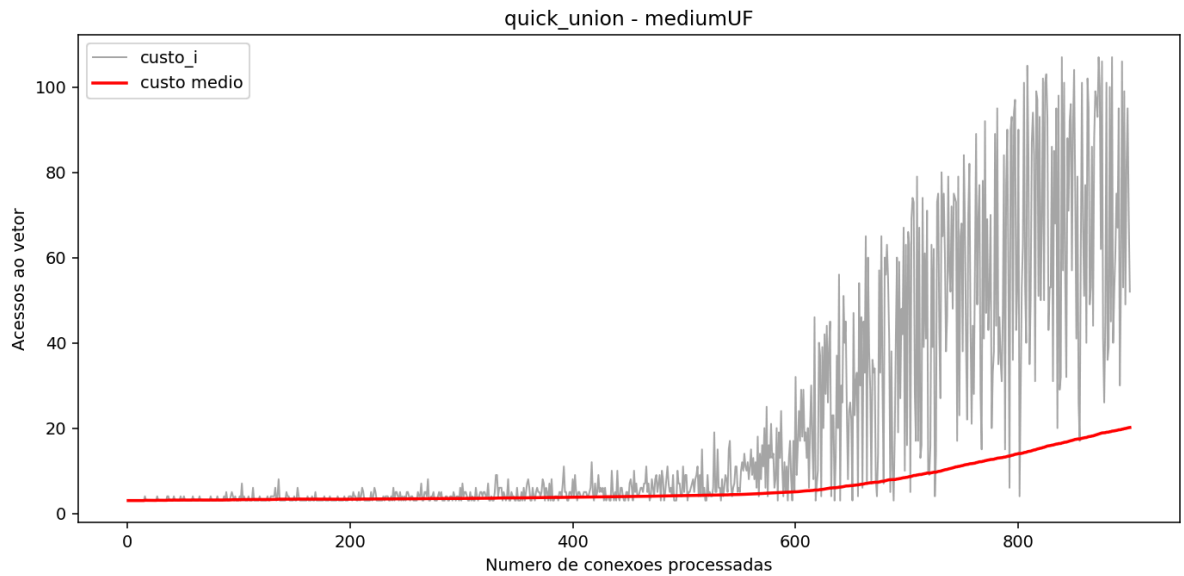
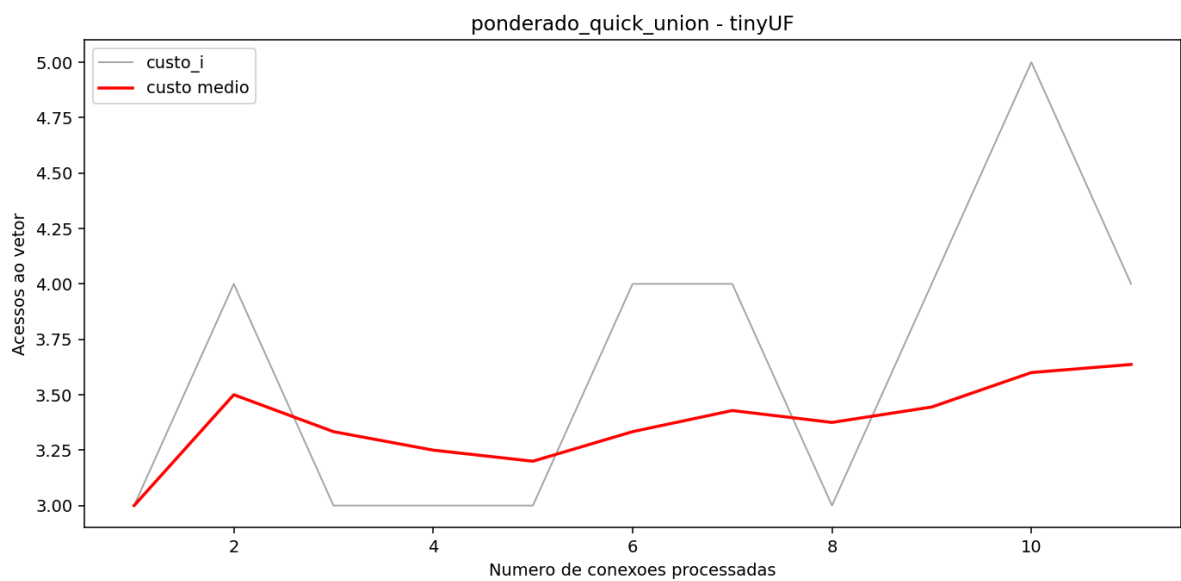


Figura 06 - [ponderado_quick_union_tinyUF_custos]



Observação: Todos os gráficos gerados estão contidos na pasta /plots do projeto,

Gráficos gerados:

- quick_find_tinyUF_custos.png
- quick_find_mediumUF_custos.png
- quick_find_teste_10000_custos.png
- quick_union_tinyUF_custos.png
- quick_union_mediumUF_custos.png

- quick_union_teste_10000_custos.png
- quick_union_teste_50000_custos.png
- quick_union_teste_100000_custos.png
- ponderado_quick_union_tinyUF_custos.png
- ponderado_quick_union_mediumUF_custos.png
- ponderado_quick_union_teste_10000_custos.png
- ponderado_quick_union_teste_50000_custos.png
- ponderado_quick_union_teste_100000_custos.png
- ponderado_quick_union_teste_500000_custos.png

3.3 Gráficos de tempo por tamanho da instância

Para cada algoritmo, foi gerado um gráfico com:

- Eixo X: quantidade de elementos N
- Eixo Y: tempo médio de execução
- Barras de erro: desvio padrão

Figura 07 - quick_find_tempo_por_n.png

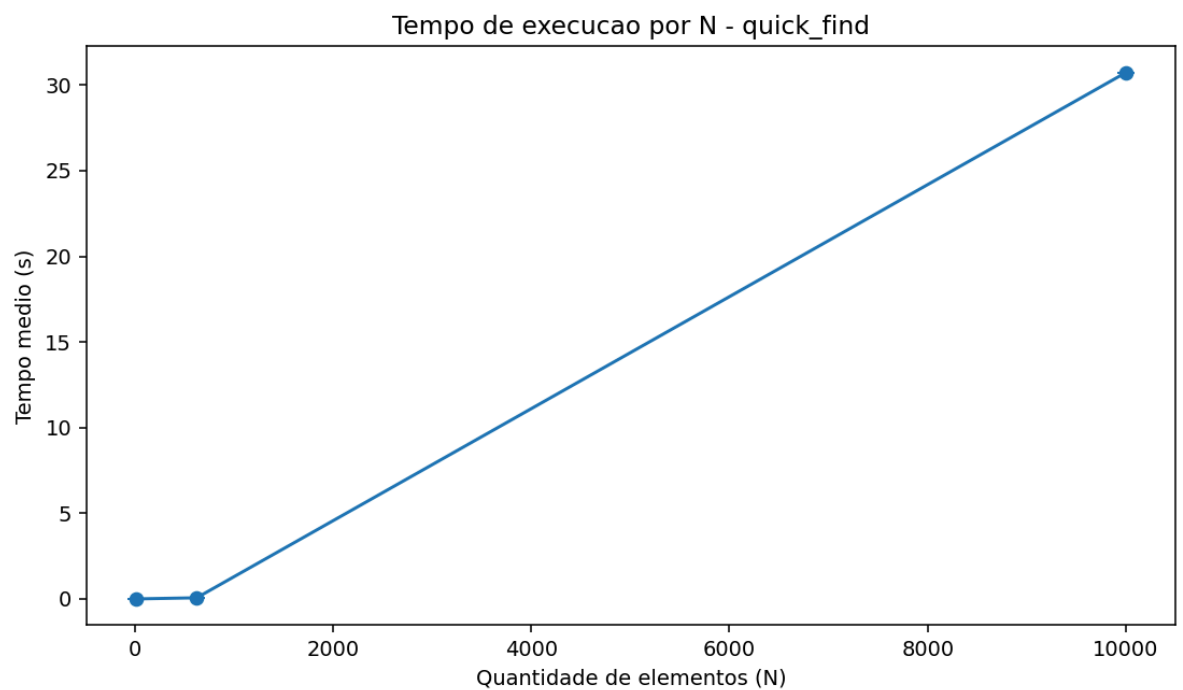
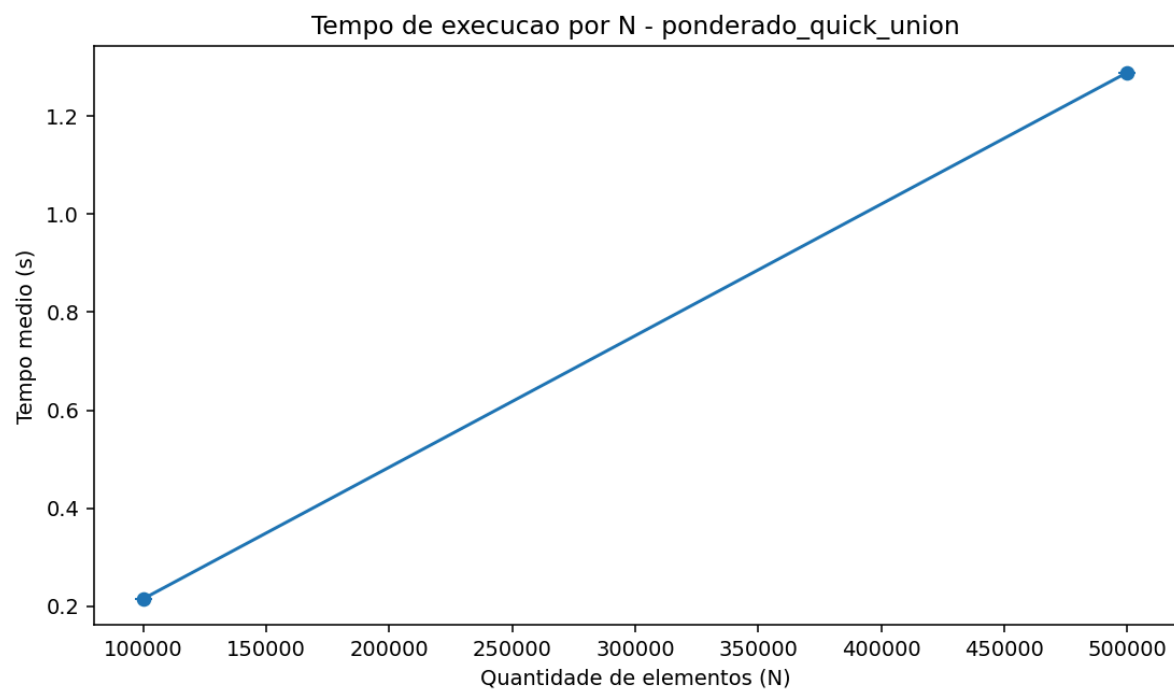


Figura 08 - quick_union_tempo_por_n.png



Figura 09 - ponderado_quick_union_tempo_por_n.png



3.4 Critério de consolidação dos resultados

O arquivo results/resumo_tempos.csv representa sempre o último lote executado. Por isso, as tabelas deste relatório distinguem:

- resultados do lote corrente (Tabela 1);
- medições de lotes anteriores (Tabela 2).

Essa separação foi mantida para evitar mistura de configurações diferentes de execução (por exemplo, valores de runs distintos).

4. DISCUSSÃO

4.1 Desempenho do Quick-Find em entradas grandes

O Quick-Find tende a apresentar desempenho inferior em entradas grandes, principalmente em padrões adversariais, pois a operação union pode exigir varredura completa do vetor id para atualizar identificadores de componentes. Esse comportamento explica o aumento significativo de custo por operação e de tempo total em instâncias de maior N.

Na medição histórica de Quick-Find em teste_10000, observou-se tempo médio de 57.788571 s, desvio padrão de 29.346822 s e custo médio final de 15002.0000, indicando custo elevado já em uma instância intermediária.

4.2 Como o Quick-Union modifica a estrutura de dados

No Quick-Union, os componentes são representados como árvores, onde id[i] indica o pai do elemento i. A operação union passa a conectar raízes, sem atualizar todos os elementos de um componente, reduzindo o custo de união em comparação ao Quick-Find para muitos cenários. No (data_lote_3), para teste_100000 com 1 repetição, o Quick-Union apresentou 1271.827457s, evidenciando custo computacional muito alto em entrada adversarial de grande escala.

4.3 Ganho do Quick-Union Ponderado

No Quick-Union Ponderado, a árvore menor é conectada a maior, o que reduz o crescimento da altura das árvores ao longo das uniões. Em geral, isso melhora o custo médio das operações find/connected e, consequentemente, o tempo total, principalmente em instâncias grandes.

No (data_lote_3), o Quick-Union Ponderado apresentou 0.215212 s em teste_100000 e 1.288780 s em teste_500000 (1 repeticao cada), mantendo vantagem expressiva em relação ao Quick-Union no mesmo perfil de entrada adversarial.

4.4 Confirmação das previsões teóricas

Os resultados experimentais devem ser comparados com a teoria:

- Quick-Find: Pior escalabilidade para uniões frequentes em N alto;
- Quick-Union: Melhora estrutural em relação ao Quick-Find, mas ainda sensível a formatos de árvore desfavoráveis;
- Quick-Union Ponderado: desempenho superior por controlar a altura das árvores.

Com base nos resultados práticos do trabalho, as observações experimentais confirmam as previsões teóricas: o Quick-Union Ponderado apresenta desempenho significativamente superior ao Quick-Union em instâncias adversariais maiores. Por outro lado, no lote corrente foi usado runs=1 para viabilizar o processamento das instâncias mais pesadas..

4.5 Limitações do estudo

As principais limitações observadas foram:

- Cobertura parcial de algoritmos por lote, devido a restrições de tempo de execução em instâncias muito grandes;
- Ausência de repetições múltiplas no lote atual (runs=1), impossibilitando estimativas mais estáveis de variabilidade;
- Consolidação em lotes independentes, exigindo cuidado na comparação direta entre resultados de campanhas distintas.

Essas limitações não invalidam a tendência observada, mas indicam que a generalização quantitativa deve ser feita com cautela.

5. CONCLUSÃO

Este trabalho apresentou a implementação e a avaliação experimental de três algoritmos Union-Find em instâncias de conectividade dinâmica com diferentes tamanhos. Os resultados ficaram alinhados com a teoria: Quick-Find tende a escalar pior em entradas adversariais; Quick-Union melhora a estruturação das uniões, mas ainda pode ter custo alto em grandes N ; e Quick-Union Ponderado mostrou desempenho claramente superior em tempo. Assim, no contexto analisado, a estratégia de ponderação se mostrou a opção mais eficiente para cenários com grande volume de conexões.

6. REFERÊNCIAS

SEGEWICK, R.; WAYNE, K. Algorithms. 4th ed. Addison-Wesley.

CORMEN, T. H. et al. Introduction to Algorithms. MIT Press.

Material de aula da disciplina: Algoritmos e Complexidade Computacional 2026, aula_03